

Code Review Summary: Matching API Endpoints

Dog Adoption Matching Platform | Session date: 2026-07-01

Files reviewed: `api/matches.py`, `api/match_action.py`, `api/dogs.py`, `api/db_client.py` (new functions: `fetch_dogs_page`, `upsert_match_status`)

Overview

Three new Vercel serverless (FastAPI) endpoints were added to expose the existing matching engine and dog catalog over HTTP: **GET /api/matches** (an adopter's valid matches, score ≥ 70), **POST /api/match_action** (confirm/reject a match), and **GET /api/dogs** (paginated dog listing). A code-reviewer subagent then audited the diff for correctness, security, and adherence to the project's backend conventions (`api/backend-rules.md`). This document summarizes what the review found, what was fixed immediately, and what remains open.

Fixed This Session

- [CRITICAL]** - `matching_score` was never persisted. `match_action.py` upserted `potential_matches` rows without a score, so every confirmed/rejected match got `matching_score = NULL` forever - silently breaking `SPEC.md`'s requirement to display the match percentage on the Confirmed/Rejected tabs. **Fix:** the endpoint now computes `matcher.calculate_match_score(dog, adopter)` server-side (never trusts a client-supplied score) and persists it via a new `matching_score` parameter on `upsert_match_status`.
- [WARNING]** - **GET /api/dogs** was unbounded when `limit` was omitted (`fetch_all_dogs()` = a full unfiltered `SELECT *`), violating `backend-rules.md`'s guidance against heavy synchronous loops over large datasets. **Fix:** `limit` now defaults to 50 (capped at 100) instead of `None`; the endpoint always paginates.
- [WARNING]** - `offset` was silently dropped when `limit` was omitted - the no-limit code branch hardcoded `offset: 0` in the response regardless of what the caller passed. **Fix:** removed that branch; pagination is now applied consistently, so `offset` is never ignored.
- [WARNING]** - **No check that a dog was still available before confirming a match** - two different adopters could both confirm the same dog, or an adopter could confirm a dog already marked `adopted`. **Fix:** confirming (not rejecting) a dog with `status != 'available'` now returns `409 Conflict`.
- [WARNING]** - `upsert_match_status`'s `match_status` parameter was a loose `str` despite the DB CHECK constraint restricting it to three values. **Fix:** tightened to `Literal["pending", "confirmed", "rejected"]`.

All fixes were verified: `ruff check` clean, `ruff format` applied, the full 35-test pytest suite still passes, and each fix was re-confirmed with a manual FastAPI `TestClient` smoke test (score now persists on confirm, unavailable dogs return 409, pagination defaults/offset behave correctly).

Open / Deferred (Not Fixed This Session)

- [HIGH]** - **No auth/ownership check on `adopter_id`**. `db_client` authenticates with the Supabase service-role key, which bypasses Row Level Security, and neither endpoint verifies the caller's session. Any `adopter_id` passed in is trusted as-is - one adopter could currently

query *or mutate* another adopter's match state. The write endpoint (`match_action.py`) is the higher-priority target here, since it lets a caller *change* another adopter's data, not just read it. Must be fixed before these routes are exposed publicly.

[MEDIUM] - GET /api/matches still doesn't persist `pending` rows. Only the confirm/reject path in `match_action.py` writes to `potential_matches` now (with a score). The future 'All Recommendations' tab (SPEC.md Section 101) will need pending rows seeded somewhere so it can read scores back from the DB instead of recomputing them on every request.

[MEDIUM] - No automated test coverage for `matches.py`, `match_action.py`, `dogs.py`, or the two new `db_client.py` functions - only manually smoke-tested this session via monkeypatched `TestClient` calls. Only `matcher.py` has a real pytest suite.

[LOW] - No pydantic response models yet - all three endpoints return `dict[str, Any]`, which loses FastAPI's automatic OpenAPI schema/response validation. Confirmed no adopter PII is leaked by current responses (`Adopter` rows are never returned to the client`).

[LOW] - `fetch\_dogs\_page` has no internal guard against `limit <= 0` if it's ever called from somewhere other than `dogs.py`'s bounded query params; and its `count="exact"` triggers a full `COUNT(*)` on every paginated request - fine at current scale, worth revisiting (`count="estimated"`) if the dogs table grows large.

[LOW] - Narrow TOCTOU race on `match_action.py`'s upsert: if a dog or adopter row is deleted between the existence pre-check and the upsert call, the failure surfaces as a generic 500 instead of a 404/409. Low priority given `ON DELETE CASCADE` semantics and low likelihood.`

Also Still Open (Project-Wide, Not From This Review)

- Migration `20260630120000_reconcile_adopter_preferences_with_spec.sql` is not confirmed applied to the live Supabase project (`jyhkikmuehqtouhmex`). `supabase migration list` shows a blank Remote column for all 4 local migrations - ambiguous, could mean genuinely unapplied or applied out-of-band via the SQL Editor. Needs explicit confirmation/action before `get_valid_matches` or the new endpoints can be trusted against live data.